



БАЗЫ ДАННЫХ

Лекция 2.

Агрегатные функции, группирование

Подзапросы

Соединение таблиц

Порядок операций в запросе

SELECT

FROM

Порядок операций в запросе

SELECT

FROM

WHERE

ORDER BY

LIMIT

Порядок операций в запросе

SELECT

FROM

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

Таблица поставщиков (s):

n_post	name	rating	town
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

Таблица товаров (p):

n_det	name	cvet	ves	town
P1	Monitor screen	Red	12000	London
P2	Computer mouse	Green	150	Paris
P3	Keyboard	Blue	400	Rome
P4	Keyboard	Red	350	London
P5	Power unit	Blue	1200	Paris
P6	Computer case	Red	7500	London

Таблица поставок (sp):

n_post	n_det	date_post	kol
S1	P1	2021-02-01	300
S1	P2	2021-04-05	200
S1	P3	2021-05-12	400
S1	P4	2021-06-15	200
S1	P5	2021-07-22	100
S1	P6	2021-08-13	100
S2	P1	2021-03-03	300
S2	P2	2021-06-12	400
S3	P2	2021-04-04	200
S4	P2	2021-03-23	200
S4	P4	2021-06-17	300
S4	P5	2021-08-22	400

Агрегатные функции

- выполняют вычисление на наборе строк
- принимают аргумент из одного столбца
- возвращают единственное результирующее значение

Основные агрегатные функции:

Функция	Описание
SUM()	Возвращает сумму значений в столбце.
AVG()	Возвращает среднее арифметическое значений в столбце.
MIN()	Возвращает минимальное значение в столбце.
MAX()	Возвращает максимальное значение в столбце.
COUNT()	Возвращает количество строк в группе.

Выдать общую сумму поставок

SELECT SUM(kol) FROM sp

Агрегатные функции

- выполняют вычисление на наборе строк
- принимают аргумент из одного столбца
- возвращают единственное результирующее значение

Основные агрегатные функции:

Функция	Описание
SUM()	Целые, вещественные числа
AVG()	Целые, вещественные числа
MIN()	Числа, даты, строки
MAX()	Числа, даты, строки
COUNT()	Любые значения

```
SELECT COUNT(*) FROM sp
```

Анализ запросов

Таблица test_table с полями (id, size) – целые числа

```
SELECT MIN(size)  
FROM test_table
```

```
SELECT size  
FROM test_table  
ORDER BY size  
LIMIT 1
```

Найти минимальное
значение поля size в таблице

Анализ запросов

В таблице 10 000 записей, поля (id, size)

```
SELECT MIN(size)
FROM test_table
```

	A-Z QUERY PLAN
	Aggregate (cost=170.00..170.01 rows=1 width=4)
	-> Seq Scan on test (cost=0.00..145.00 rows=10000 width=4)
	Planning Time: 0.084 ms
	Execution Time: 0.944 ms

```
SELECT size
FROM test_table
ORDER BY size
LIMIT 1
```

	A-Z QUERY PLAN
1	Limit (cost=195.00..195.00 rows=1 width=4)
2	-> Sort (cost=195.00..220.00 rows=10000 width=4)
3	Sort Key: size
4	Sort Method: top-N heapsort Memory: 262144 bytes
5	-> Seq Scan on test (cost=0.00..145.00 rows=10000 width=4)
6	Planning Time: 0.062 ms
7	Execution Time: 0.939 ms

Стоимость запроса — это абстрактная условная величина, которую СУБД использует для оценки ресурсов (процессорного времени и операций ввода-вывода), нужных для выполнения.

Анализ запросов

В таблице 10 000 записей, поля (id, size)

```
SELECT MIN(size)
FROM test_table
```

A-Z QUERY PLAN	
	Aggregate (cost=170.00..170.01 rows=1 width=4)
	-> Seq Scan on test (cost=0.00..145.00 rows=10000 width=4)
	Planning Time: 0.084 ms
	Execution Time: 0.944 ms

```
SELECT size
FROM test_table
ORDER BY size
LIMIT 1
```

A-Z QUERY PLAN	
1	Limit (cost=195.00..195.00 rows=1 width=4)
2	-> Sort (cost=195.00..220.00 rows=10000 width=4)
3	Sort Key: size
4	Sort Method: top-N heapsort Memory: 262144 bytes
5	-> Seq Scan on test (cost=0.00..145.00 rows=10000 width=4)
6	Planning Time: 0.062 ms
7	Execution Time: 0.939 ms

Стоимость запроса — это абстрактная условная величина, которую СУБД использует для оценки ресурсов (процессорного времени и операций ввода-вывода), нужных для выполнения.

Анализ запросов

В таблице 1 000 000 записей, поля (id, size)

```
SELECT MIN(size)
FROM test_table
```

```
SELECT size
FROM test_table
ORDER BY size
LIMIT 1
```

A-Z QUERY PLAN	
1	Finalize Aggregate (cost=10633.55..10633.55 rows=1 width=4)
2	-> Gather (cost=10633.33..10633.54 rows=2 width=4) (actual time=0.000..0.000 rows=2)
3	Workers Planned: 2
4	Workers Launched: 2
5	-> Partial Aggregate (cost=9633.33..9633.34 rows=1 width=4)
6	-> Parallel Seq Scan on test (cost=0.00..8591.67 rows=1000000)
7	Planning Time: 0.090 ms
8	Execution Time: 186.854 ms

A-Z QUERY PLAN	
1	Limit (cost=11675.03..11675.14 rows=1 width=4) (actual time=68.900..68.900 rows=1)
2	-> Gather Merge (cost=11675.03..108904.11 rows=833334 width=4)
3	Workers Planned: 2
4	Workers Launched: 2
5	-> Sort (cost=10675.00..11716.67 rows=416667 width=4) (actual time=68.900..68.900 rows=1)
6	Sort Key: size
7	Sort Method: top-N heapsort Memory: 25kB
8	Worker 0: Sort Method: top-N heapsort Memory: 25kB
9	Worker 1: Sort Method: top-N heapsort Memory: 25kB
10	-> Parallel Seq Scan on test (cost=0.00..8591.67 rows=416667)
11	Planning Time: 0.086 ms
12	Execution Time: 93.705 ms

EXPLAIN [ANALYZE]
select ...

План запроса – зависит от
собранной ранее
статистики

VACUUM ANALYZE таблица
– обновление статистики

Оператор GROUP BY – группирование

- Группирует таблицу таким образом, чтобы в каждой группе все строки имели одно и тоже значение поля, по которому происходит группирование
- Далее к каждой группе перекомпанованной таблицы (а не к каждой строке исходной таблицы) применяется **SELECT**, то есть каждое выражение во фразе **SELECT** принимает единственное значение для группы.
- Группирование имеет смысл с агрегатными функциями

Оператор GROUP BY – группирование

Синтаксис запроса:

```
SELECT поле_группировки, агрегатная_функция  
FROM таблица  
GROUP BY поле_группировки
```

Выдать для каждого поставляемого товара его номер и общий объем поставок:

```
SELECT n_det, SUM(kol) FROM sp  
GROUP BY n_det
```

n_det	sum(kol)
P1	300
P2	800
P4	300
P5	400

Не путать!

GROUP BY

Группирование

ORDER BY

Сортировка

Не путать!

GROUP BY

Группирование

Каждая группа ровно 1 раз

ORDER BY

Сортировка

select **DISTINCT**

Вывод уникальных записей

Оператор HAVING

- Используется для того, чтобы исключать (**фильтровать**) **группы** (аналогично WHERE для исключения строк)
- Выражение (условие) во фразе **HAVING** должно принимать единственное значение для группы, то есть
- Задает **условие с агрегатной функцией**
- Используется **ТОЛЬКО** после группирования

Группирование и HAVING

Синтаксис запроса:

```
SELECT поле_группировки, агрегатная_функция  
FROM таблица  
GROUP BY поле_группировки  
HAVING условие с агрегатной_функцией
```

Выдать номера товаров, поставляемых более чем 1 поставщиком:

```
SELECT n_det  
FROM SP  
GROUP BY n_det  
HAVING COUNT(*) > 1
```

Порядок операций в запросе

1. SELECT

2. FROM

3. WHERE (фильтр до группирования)

4. GROUP BY (группирование)

5. HAVING (фильтр после группирования)

6. ORDER BY

7. LIMIT



БАЗЫ ДАННЫХ

Лекция 2.

Подзапросы

Подзапросы

Подзапрос – оператор **SELECT**, вложенный в спецификатор **WHERE** другого оператора **SELECT** (или **INSERT, DELETE, UPDATE**).

- В состав каждого подзапроса должны входить **SELECT** и **FROM**.
- Каждый подзапрос должен быть заключен в круглые скобки, чтобы указать серверу баз данных на то, что эту операцию следует выполнить первой.

Подзапросы

Подзапросы бывают коррелированными и некоррелированными.

- Подзапрос является **коррелированным**, если его значение зависит от значения, производимого внешним оператором **SELECT**, который содержит этот подзапрос.
- Любой другой вид запроса называется некоррелированным.

Подзапросы

- **Коррелированный** подзапрос: зависит от значения результата внешнего оператора SELECT, поэтому должен выполняться повторно по одному разу для каждого значения, производимого внешним оператором SELECT.
- **Некоррелированный** подзапрос выполняется только один раз.

Подзапросы

```
SELECT * FROM s WHERE n_post IN  
(SELECT n_post FROM sp where kol>200)
```

```
SELECT * FROM s WHERE  
(SELECT kol FROM sp where sp.n_post = s.n_post)>200
```

Подзапросы

```
SELECT * FROM s WHERE n_post IN  
(SELECT n_post FROM sp where kol>200)
```

Некоррелированный подзапрос, выполнится 1 раз

```
SELECT * FROM s WHERE  
(SELECT kol FROM sp where sp.n_post = s.n_post)>200
```

Подзапросы

```
SELECT * FROM s WHERE n_post IN  
(SELECT n_post FROM sp where kol>200)
```

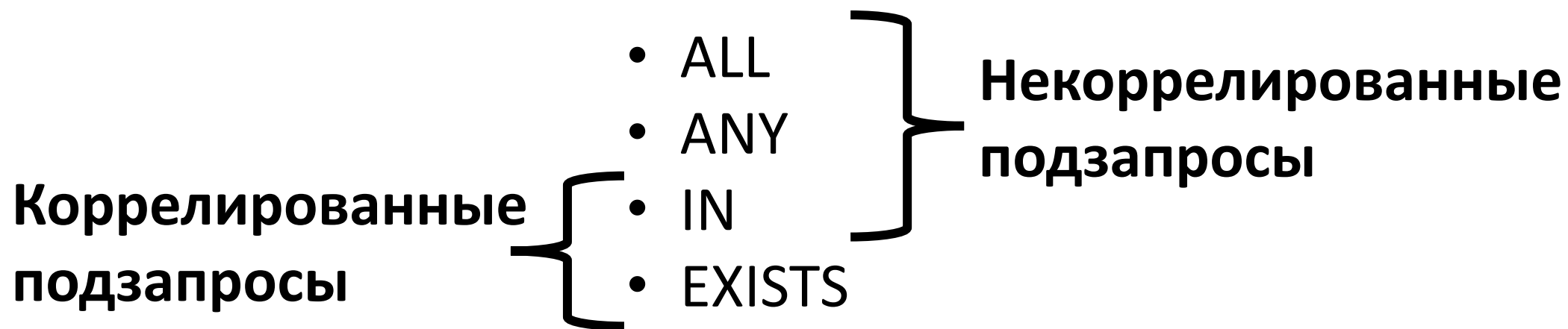
Некоррелированный подзапрос, выполнится 1 раз

```
SELECT * FROM s WHERE  
(SELECT kol FROM sp where sp.n_post = s.n_post)>200
```

**Коррелированный подзапрос, выполнится много раз
для каждой строки **внешней** таблицы**

Подзапросы

Подзапрос может включаться в спецификатор ***where*** с помощью следующих ключевых слов:



Ключевое слово ALL – некоррелированные подзапросы

- Используется перед подзапросом для определения того, выполняется ли условие сравнения **для каждого возвращаемого подзапросом значения**.
- Если подзапрос не возвращает ни одного значения, то условие поиска считается выполненным.

Получить перечень поставщиков, рейтинг которых выше рейтинга любого лондонского поставщика.

```
SELECT x.n_post, x.name, x.rating
FROM s x WHERE x.rating > ALL
  (SELECT y.rating
   FROM s y
   WHERE y.town='London')
```

n_post	name	rating
S3	Blake	30
S5	Adams	30

Ключевое слово ANY – некоррелированные подзапросы

- Используется перед запросом для определения того, выполняется ли сравнение по **крайней мере для одного значения**, возвращаемого подзапросом.
- Если подзапрос не возвращает ни одного значения, то условие поиска считается не выполненным.

Получить перечень поставщиков, рейтинг которых выше рейтинга хотя бы одного парижского поставщика.

```
SELECT x.n_post, x.name, x.rating  
FROM S x  
WHERE x.rating > ANY  
    (SELECT y.rating  
     FROM S y  
     WHERE y.town = 'Paris')
```

n_post	name	rating
S1	Smith	20
S3	Blake	30
S4	Clark	20
S5	Adams	30

Ключевое слово IN - некоррелированные подзапросы

Простой подзапрос.

Выдать фамилии поставщиков, поставляющих товар под номером P2.

```
SELECT name  
FROM s  
WHERE n_post IN  
      (SELECT n_post  
      FROM sp  
      WHERE n_det = 'P2')
```

Ключевое слово IN - некоррелированные подзапросы

Подзапрос с несколькими уровнями вложенности

Выдать фамилии поставщиков, поставляющих по крайней мере один красный товар.

```
SELECT name FROM s  
WHERE n_post IN  
      (SELECT n_post FROM sp  
        WHERE n_det IN  
              (SELECT n_det FROM p  
                WHERE cvet='Red'))
```

Ключевое слово IN - некоррелированные подзапросы

Использование одной и той же таблицы в подзапросе внешнем запросе

Выдать номера поставщиков, поставляющих, по крайней мере, один товар, поставляемый поставщиком S2.

```
SELECT DISTINCT n_post FROM sp spx  
WHERE spx.n_det IN  
  (SELECT spy.n_det FROM sp spy  
   WHERE spy.n_post='S2')
```

Некоррелированные подзапросы

Подзапрос с оператором сравнения, отличным от IN

Выдать номера поставщиков, находящихся в том же городе, что и поставщик S1

```
SELECT n_post  
FROM s  
WHERE town =  
    (SELECT town  
    FROM s  
    WHERE n_post = 'S1')
```

Ключевое слово IN - коррелированные подзапросы

Выдать фамилии поставщиков, поставляющих товар P2.

```
SELECT name  
FROM s  
WHERE 'P2' IN  
    (SELECT n_det  
        FROM SP  
        WHERE n_post = s.n_post)
```

В коррелированном подзапросе внутренний подзапрос не может быть отработан раз и навсегда, прежде чем будет отработан внешний запрос, поскольку этот внутренний подзапрос зависит от переменной, значение которой изменяется по мере того, как система проверяет различные строки таблицы, участвующие во внешнем запросе.

Ключевое слово IN - коррелированные подзапросы

Коррелированный подзапрос с использованием одной и той же таблицы

Выдать номера товаров, поставляемых более чем одним поставщиком.

```
SELECT DISTINCT spx.n_det
FROM sp spx
WHERE spx.n_det in
    (SELECT spy.n_det
     FROM sp spy
     WHERE spy.n_post<>spx.n_post)
```

Оператор EXISTS – коррелированные подзапросы

Выдать фамилии поставщиков, поставляющих товара P2.

```
SELECT name  
FROM s  
WHERE EXISTS  
    (SELECT * from sp  
     WHERE sp.n_post = s.n_post and n_det = 'P2')
```

Использование функций в подзапросе

Выдать поставщиков с рейтингом меньшим, чем максимальный в таблице

```
SELECT name  
FROM s  
WHERE rating < (SELECT MAX(rating) FROM s)
```

Выдать информацию о всех поставщиках, у которых рейтинг больше, чем средний для конкретно их города

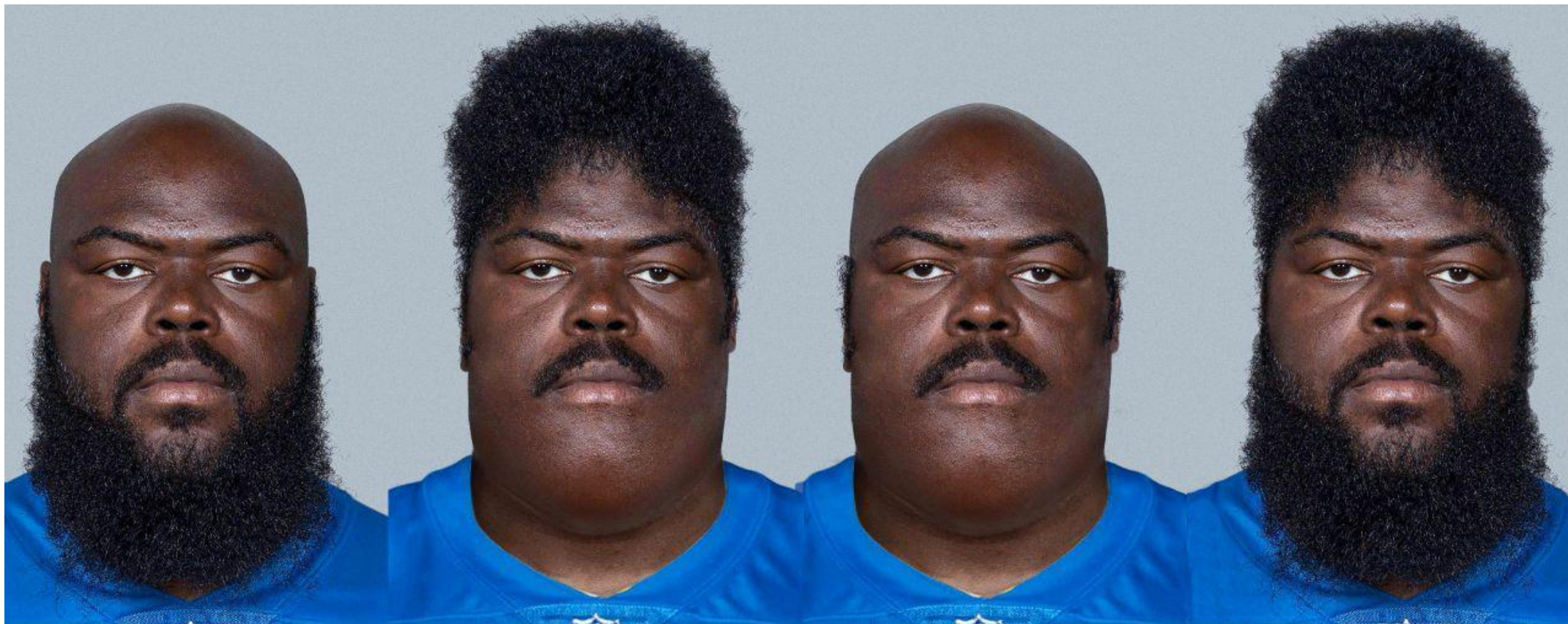
```
SELECT * FROM s sx  
WHERE rating >  
  (SELECT AVG(rating)  
   FROM s sy  
   WHERE sy.town=sx.town)
```



БАЗЫ ДАННЫХ

Лекция 2.

Соединение таблиц



LEFT JOIN

RIGHT JOIN

INNER JOIN

FULL OUTER JOIN

Соединение таблиц

Оператор JOIN

1. Внутреннее соединение INNER JOIN
2. Внешнее соединение:
 - левое LEFT [OUTER] JOIN
 - правое RIGHT [OUTER] JOIN
 - полное FULL [OUTER] JOIN
3. Декартовое произведение CROSS JOIN
4. Естественное соединение NATURAL JOIN

Декартово произведение – CROSS JOIN

- Каждая запись из первой таблицы соединяется с каждой записью из второй таблицы
- Не требует условия соединения
- ```
SELECT *
FROM таблица1
CROSS JOIN таблица2
```

**Таблица поставщиков (s):**

| n_post | name  | rating | town   |
|--------|-------|--------|--------|
| S1     | Smith | 20     | London |
| S2     | Jones | 10     | Paris  |
| S3     | Blake | 30     | Paris  |
| S4     | Clark | 20     | London |
| S5     | Adams | 30     | Athens |

**Таблица товаров (p):**

| n_det | name           | cvet  | ves   | town   |
|-------|----------------|-------|-------|--------|
| P1    | Monitor screen | Red   | 12000 | London |
| P2    | Computer mouse | Green | 150   | Paris  |
| P3    | Keyboard       | Blue  | 400   | Rome   |
| P4    | Keyboard       | Red   | 350   | London |
| P5    | Power unit     | Blue  | 1200  | Paris  |
| P6    | Computer case  | Red   | 7500  | London |

**Таблица поставок (sp):**

| n_post | n_det | date_post  | kol |
|--------|-------|------------|-----|
| S1     | P1    | 2021-02-01 | 300 |
| S1     | P2    | 2021-04-05 | 200 |
| S1     | P3    | 2021-05-12 | 400 |
| S1     | P4    | 2021-06-15 | 200 |
| S1     | P5    | 2021-07-22 | 100 |
| S1     | P6    | 2021-08-13 | 100 |
| S2     | P1    | 2021-03-03 | 300 |
| S2     | P2    | 2021-06-12 | 400 |
| S3     | P2    | 2021-04-04 | 200 |
| S4     | P2    | 2021-03-23 | 200 |
| S4     | P4    | 2021-06-17 | 300 |
| S4     | P5    | 2021-08-22 | 400 |

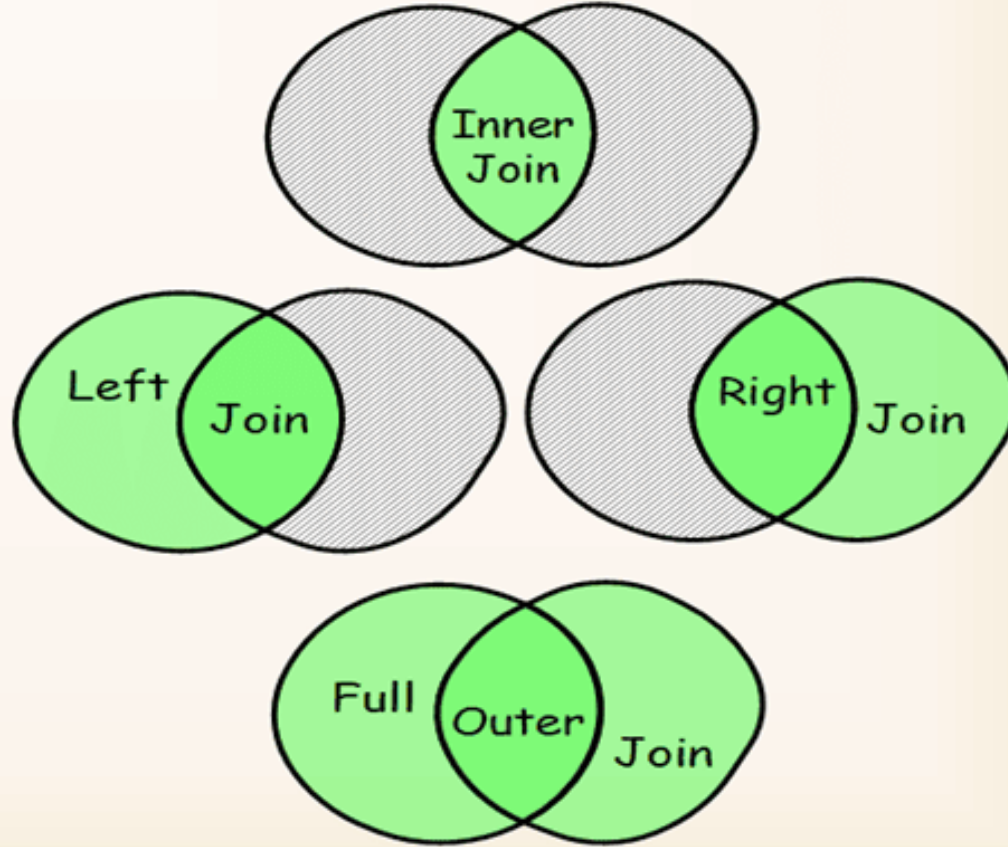
Выдать информацию обо  
всех возможных парах  
поставщик - товар.

```
SELECT s.*, p.*
FROM s
CROSS JOIN p
```

| n_post | name  | rating | town   | n_det | name           | cvet  | ves   | town   |
|--------|-------|--------|--------|-------|----------------|-------|-------|--------|
| S1     | Smith | 20     | London | P1    | Monitor screen | Red   | 12000 | London |
| S2     | Jones | 10     | Paris  | P1    | Monitor screen | Red   | 12000 | London |
| S3     | Blake | 30     | Paris  | P1    | Monitor screen | Red   | 12000 | London |
| S4     | Clark | 20     | London | P1    | Monitor screen | Red   | 12000 | London |
| S5     | Adams | 30     | Athens | P1    | Monitor screen | Red   | 12000 | London |
| S1     | Smith | 20     | London | P2    | Computer mouse | Green | 150   | Paris  |
| S2     | Jones | 10     | Paris  | P2    | Computer mouse | Green | 150   | Paris  |
| S3     | Blake | 30     | Paris  | P2    | Computer mouse | Green | 150   | Paris  |
| S4     | Clark | 20     | London | P2    | Computer mouse | Green | 150   | Paris  |
| S5     | Adams | 30     | Athens | P2    | Computer mouse | Green | 150   | Paris  |
| S1     | Smith | 20     | London | P3    | Keyboard       | Blue  | 400   | Rome   |
| S2     | Jones | 10     | Paris  | P3    | Keyboard       | Blue  | 400   | Rome   |
| S3     | Blake | 30     | Paris  | P3    | Keyboard       | Blue  | 400   | Rome   |
| S4     | Clark | 20     | London | P3    | Keyboard       | Blue  | 400   | Rome   |
| S5     | Adams | 30     | Athens | P3    | Keyboard       | Blue  | 400   | Rome   |
| S1     | Smith | 20     | London | P4    | Keyboard       | Red   | 350   | London |
| S2     | Jones | 10     | Paris  | P4    | Keyboard       | Red   | 350   | London |
| S3     | Blake | 30     | Paris  | P4    | Keyboard       | Red   | 350   | London |
| S4     | Clark | 20     | London | P4    | Keyboard       | Red   | 350   | London |
| S5     | Adams | 30     | Athens | P4    | Keyboard       | Red   | 350   | London |
| S1     | Smith | 20     | London | P5    | Power unit     | Blue  | 1200  | Paris  |
| S2     | Jones | 10     | Paris  | P5    | Power unit     | Blue  | 1200  | Paris  |
| S3     | Blake | 30     | Paris  | P5    | Power unit     | Blue  | 1200  | Paris  |
| S4     | Clark | 20     | London | P5    | Power unit     | Blue  | 1200  | Paris  |
| S5     | Adams | 30     | Athens | P5    | Power unit     | Blue  | 1200  | Paris  |
| S1     | Smith | 20     | London | P6    | Computer case  | Red   | 7500  | London |
| S2     | Jones | 10     | Paris  | P6    | Computer case  | Red   | 7500  | London |
| S3     | Blake | 30     | Paris  | P6    | Computer case  | Red   | 7500  | London |
| S4     | Clark | 20     | London | P6    | Computer case  | Red   | 7500  | London |
| S5     | Adams | 30     | Athens | P6    | Computer case  | Red   | 7500  | London |

Table A

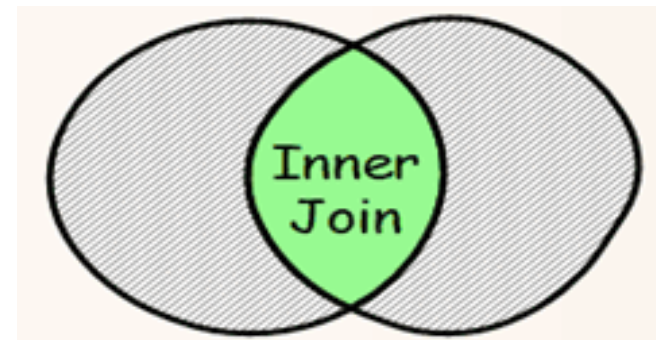
Table B



# Внутреннее соединение таблиц – INNER JOIN

- Используется, когда нужно включить все **строки** из обеих таблиц, **удовлетворяющие условию** объединения, и **только их**
- В WHERE сравниваются значения полей из разных таблиц:
  - поля имеют один и тот же тип данных
  - могут иметь разные имена

- Упрощенно на кругах Эйлера:



# Внутреннее соединение таблиц – INNER JOIN

- SELECT \*  
FROM *таблица1*  
INNER JOIN *таблица2*  
ON *условие*

или

- SELECT \*  
FROM *таблица1*  
INNER JOIN *таблица2*  
USING (*поле*)

# Внутреннее соединение таблиц – INNER JOIN

- ```
SELECT *  
FROM таблица1  
INNER JOIN таблица2  
ON условие
```



```
SELECT s.n_post , p.n_det  
FROM s  
INNER JOIN p  
ON s.town=p.town
```

или

- ```
SELECT *
FROM таблица1
INNER JOIN таблица2
USING (поле)
```

```
SELECT s.n_post , p.n_det
FROM s
INNER JOIN p
USING (town)
```

# Внутреннее соединение таблиц – INNER JOIN

*2 поля:*  
*s.town,*  
*p.town*

```
SELECT s.n_post , p.n_det
FROM s
INNER JOIN p
ON s.town=p.town
```

*1 поле:*  
*town*

```
SELECT s.n_post , p.n_det
FROM s
INNER JOIN p
USING (town)
```

# Внутреннее соединение таблиц – INNER JOIN

*2 поля:* `SELECT s.n_post , p.n_det`  
*s.town,* `FROM s`  
*p.town* `INNER JOIN p`  
`ON s.town=p.town`

*Любые поля одного типа*

*1 поле:* `SELECT s.n_post , p.n_det`  
*town* `FROM s`  
`INNER JOIN p`  
`USING (town)`

*Только одноименные поля*

# Внутреннее соединение таблиц – INNER JOIN

- Строится **декартово произведение** строк первой и второй таблиц
- Из полученного набора данных **отбираются** записи, удовлетворяющие условиям объединения.
- Блоки данных из двух таблиц объединяются, как только в указанных полях будут найдены совпадающие значения.

В РЕАЛЬНОСТИ НЕ СОВСЕМ ТАК, а более оптимизированно!!!!

# Стандарт ANSI SQL



## Стандарт ANSI

*SELECT s.\*, p.\**

*FROM s*

*CROSS JOIN p*

*SELECT s.n\_post, p.n\_det*

*FROM s*

*INNER JOIN p*

*ON s.town=p.town*

## Отклонение от стандарта (не рекомендуется)

*SELECT s.\*, p.\**

*FROM s, p*

*SELECT s.n\_post, p.n\_det*

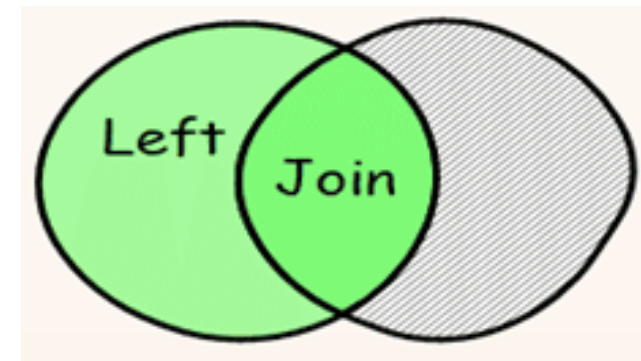
*FROM s, p*

*WHERE s.town=p.town*

# Левое внешнее соединение таблиц – LEFT JOIN

- Содержит как минимум один экземпляр каждой строки из таблицы, указанной **слева** от слова JOIN.
- Содержит строки из правой таблицы, удовлетворяющие условию (WHERE)
- Отсутствующие поля из правой таблицы заполняются значениями NULL.

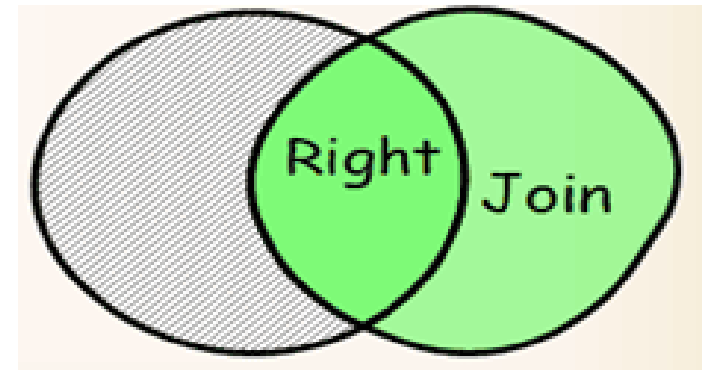
- Упрощенно на кругах Эйлера:



# Правое внешнее соединение таблиц – RIGHT JOIN

- Содержит как минимум один экземпляр каждой строки из таблицы, указанной **справа** от слова JOIN.
- Содержит строки из левой таблицы, удовлетворяющие условию (WHERE)
- Отсутствующие поля из левой таблицы заполняются значениями NULL.

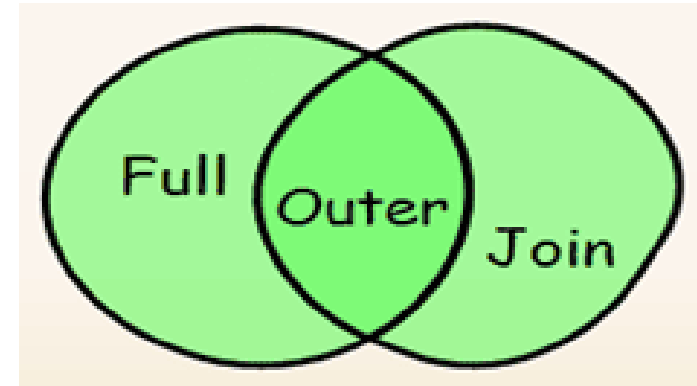
- Упрощенно на кругах Эйлера:



# Полное внешнее соединение таблиц – FULL JOIN

- Содержит как минимум один экземпляр каждой строки из **каждой** таблицы.
- В строках, удовлетворяющих условию, заполняются все поля
- Отсутствующие поля в записях результирующего набора заполняются значениями NULL.

- Упрощенно на кругах Эйлера:



# Внешнее соединение таблиц

```
SELECT *
FROM таблица1
LEFT / RIGHT / FULL JOIN таблица2
ON условие / USING (поле)
```

В некоторых СУБД RIGHT и FULL JOIN отсутствуют вовсе

# Внешнее соединение таблиц

```
SELECT s.n_post, s.name, sp.kol
FROM s
LEFT JOIN sp ON s.n_post = sp.n_post
```

| n_post | name  | kol  |
|--------|-------|------|
| S1     | Smith | 300  |
| S1     | Smith | 200  |
| S1     | Smith | 400  |
| S1     | Smith | 200  |
| S1     | Smith | 100  |
| S1     | Smith | 100  |
| S2     | Jones | 300  |
| S2     | Jones | 400  |
| S3     | Blake | 200  |
| S4     | Clark | 200  |
| S4     | Clark | 300  |
| S4     | Clark | 400  |
| S5     | Adams | NULL |

# Естественное соединение таблиц – NATURAL JOIN



- Соединение по равенству **ВСЕХ** одноименных полей
- Из результатов исключается по одному экземпляру каждого общего атрибута
- Не требует условия (так как оно формируется «само»)

```
SELECT *FROM s
NATURAL JOIN sp
```

*//внутреннее соединение по общему полю n\_post*

# Соединение таблиц

## Оператор JOIN

1. Внутреннее соединение INNER JOIN
2. Внешнее соединение:
  - левое LEFT [OUTER] JOIN
  - правое RIGHT [OUTER] JOIN
  - полное FULL [OUTER] JOIN
3. Декартово произведение CROSS JOIN
4. Естественное соединение NATURAL JOIN

## Соединение множества таблиц

```
SELECT DISTINCT s.town, p.town
FROM sp
INNER JOIN s ON s.n_post = sp.n_post
INNER JOIN p ON p.n_det = sp.n_det
```

# Соединение множества таблиц

```
SELECT DISTINCT s.town, p.town
FROM sp
INNER JOIN s ON s.n_post = sp.n_post
INNER JOIN p ON p.n_det = sp.n_det
```

**INNER JOIN работает по умолчанию**

```
SELECT DISTINCT s.town, p.town
FROM sp
JOIN s ON s.n_post = sp.n_post
JOIN p ON p.n_det = sp.n_det
```

# Соединение с дополнительным условием

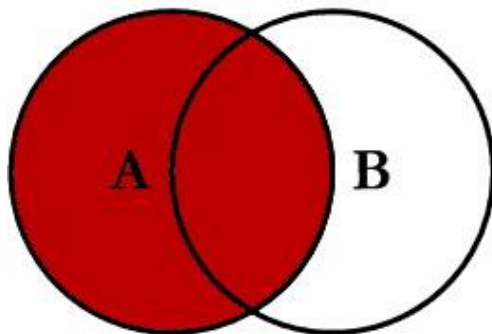
Выдать все комбинации информации о поставщиках и товарах, расположенных в одном городе, кроме поставщиков с рейтингом = 20.

```
SELECT s.n_post, p.n_det, rating
FROM s
CROSS JOIN p
WHERE s.town = p.town AND s.rating < > 20
```

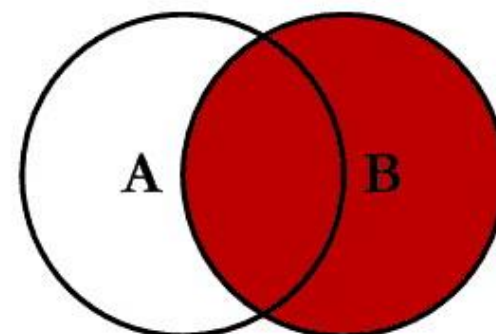
| n_post | n_det | rating |
|--------|-------|--------|
| S2     | P2    | 10     |
| S3     | P2    | 30     |
| S2     | P5    | 10     |
| S3     | P5    | 30     |

# SQL JOINS

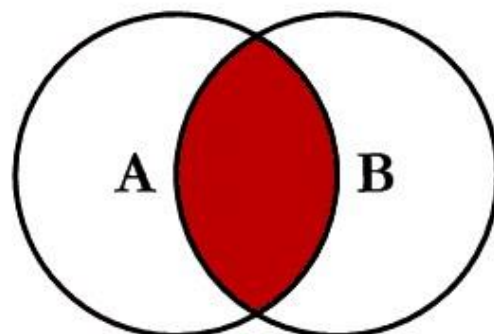
(упрощенно, но наглядно)



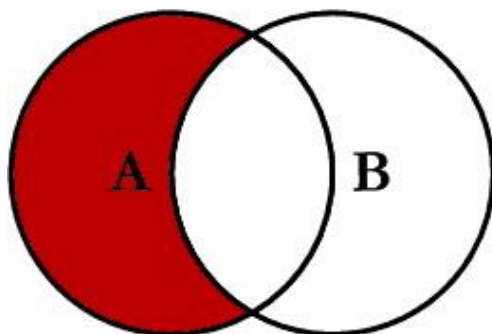
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



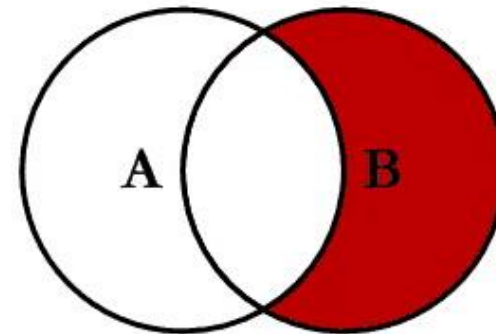
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



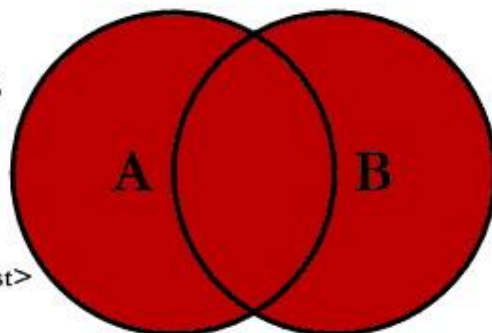
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



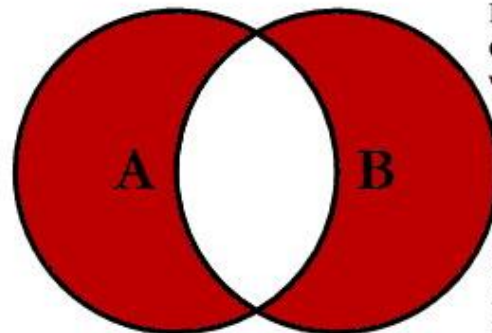
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

# JOIN – это не круги Эйлера!!!

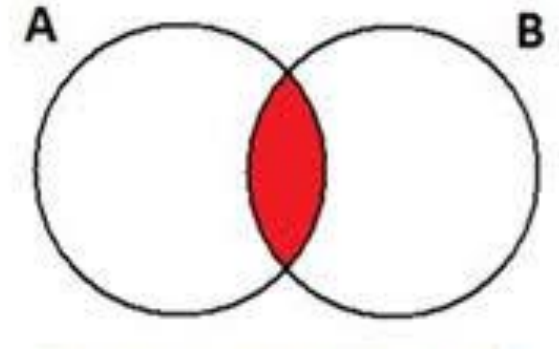
users

| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Alice   |
| 4  | Charlie |

attackers

| id | name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Eva   |

*Пересечение  
по именам*



# JOIN – это не круги Эйлера!!!

users

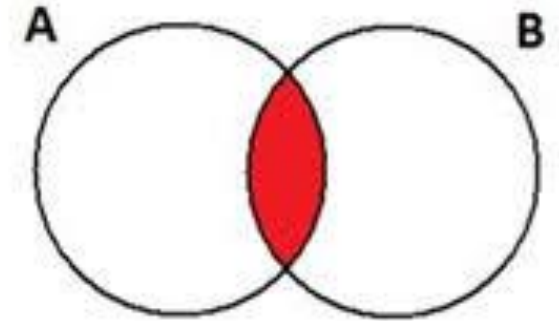
| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Alice   |
| 4  | Charlie |

attackers

| id | name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Eva   |

*Пересечение  
по именам*

Alice, Bob



# JOIN – это не круги Эйлера!!!

users

| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Alice   |
| 4  | Charlie |

attackers

| id | name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Eva   |

*INNER JOIN по именам*

| u.id | u.name  | a.id | a.name |
|------|---------|------|--------|
| 1    | Alice   | 1    | Alice  |
| 2    | Bob     | 1    | Alice  |
| 3    | Alice   | 1    | Alice  |
| 4    | Charlie | 1    | Alice  |
| 1    | Alice   | 2    | Bob    |
| 2    | Bob     | 2    | Bob    |
| 3    | Alice   | 2    | Bob    |
| 4    | Charlie | 2    | Bob    |
| 1    | Alice   | 3    | Eva    |
| 2    | Bob     | 3    | Eva    |
| 3    | Alice   | 3    | Eva    |
| 4    | Charlie | 3    | Eva    |

# JOIN – это не круги Эйлера!!!

users

| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Alice   |
| 4  | Charlie |

attackers

| id | name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Eva   |

*INNER JOIN по именам*

| u.id | u.name  | a.id | a.name |
|------|---------|------|--------|
| 1    | Alice   | 1    | Alice  |
| 2    | Bob     | 1    | Alice  |
| 3    | Alice   | 1    | Alice  |
| 4    | Charlie | 1    | Alice  |
| 1    | Alice   | 2    | Bob    |
| 2    | Bob     | 2    | Bob    |
| 3    | Alice   | 2    | Bob    |
| 4    | Charlie | 2    | Bob    |
| 1    | Alice   | 3    | Eva    |
| 2    | Bob     | 3    | Eva    |
| 3    | Alice   | 3    | Eva    |
| 4    | Charlie | 3    | Eva    |

# JOIN – это не круги Эйлера!!!

users

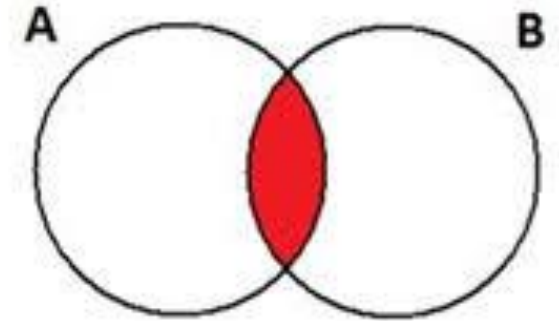
| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Alice   |
| 4  | Charlie |

attackers

| id | name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Eva   |

*Пересечение  
по именам*

Alice, Bob



*INNER JOIN*

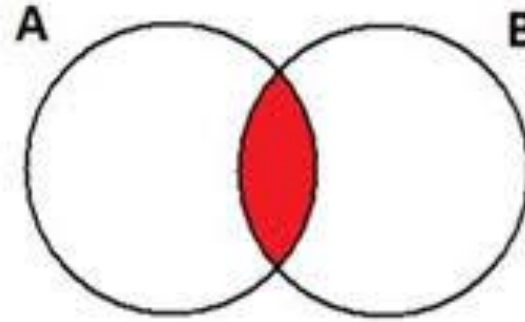
*по именам*

| u.id | u.name | a.id | a.name |
|------|--------|------|--------|
| 1    | Alice  | 1    | Alice  |
| 3    | Alica  | 1    | Alice  |
| 2    | Bob    | 2    | Bob    |

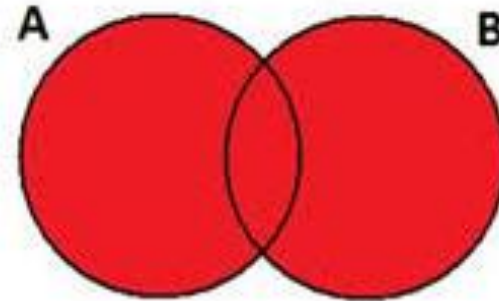
# Операторы на таблицах

## 1. Объединение:

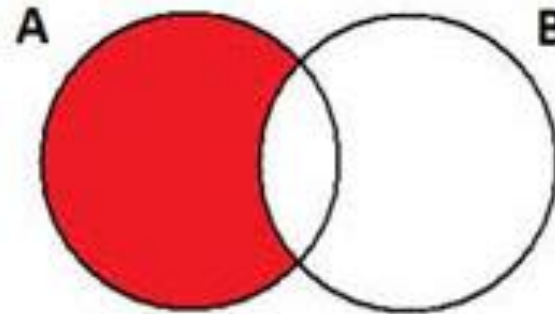
- без дубликатов UNION
- с дубликатами UNION ALL



## 2. Пересечение INTERSECT



## 3. Разность EXCEPT



# Объединение таблиц – UNION [ALL]

- Объединяемые таблицы должны быть **совместны** по объединению:
  - иметь одинаковое число столбцов;
  - соответствующие столбцы должны иметь одинаковые типы.
- Любое число предложений SELECT может быть соединено оператором UNION.

*(SELECT n\_det FROM p WHERE weight > 16)*

***UNION [ALL]***

*(SELECT n\_det FROM p WHERE n\_post > 'S2')*

# Операторы INTERSECT и EXCEPT

- Синтаксис и требования аналогичны оператору UNION

*(SELECT n\_det FROM p WHERE weight > 16)*

***INTERSECT / EXCEPT***

*(SELECT n\_det FROM p WHERE n\_post > 'S2')*

- UNION ALL – объединение с дубликатами – объединяет две выборки, сохраняя все строки (работает быстрее, чем UNION).
- INTERSECT – пересечение – возвращает общие строки из двух выборок.
- EXCEPT (MINUS в некоторых СУБД) – разность – возвращает записи из первой выборки, которых нет во второй.

# Отличия JOIN от UNION

## JOIN

- соединяет таблицы по «горизонтали»
- часто есть условие соединения
- результирующая таблица содержит больше полей, чем исходная
- результирующая таблица может содержать больше, меньше или столько же записей, как и исходные

| n_post | name  | rating | town   |
|--------|-------|--------|--------|
| S1     | Smith | 20     | London |
| S2     | Jones | 10     | Paris  |
| S3     | Blake | 30     | Paris  |
| S4     | Clark | 20     | London |
| S5     | Adams | 30     | Athens |

| n_det | name           | cvet  | ves   | town   |
|-------|----------------|-------|-------|--------|
| P1    | Monitor screen | Red   | 12000 | London |
| P2    | Computer mouse | Green | 150   | Paris  |
| P3    | Keyboard       | Blue  | 400   | Rome   |
| P4    | Keyboard       | Red   | 350   | London |
| P5    | Power unit     | Blue  | 1200  | Paris  |
| P6    | Computer case  | Red   | 7500  | London |

# Отличия JOIN от UNION

## UNION

- соединяет выборки по «вертикали»
- нет условия, поля должны полностью совпадать
- результирующая таблица столько же больше полей, как исходная
- результирующая таблица содержит больше записей, чем исходные

| town   |
|--------|
| London |
| Paris  |
| Rome   |
| London |
| Paris  |
| London |

| town   |
|--------|
| London |
| Paris  |
| Paris  |
| London |
| Athens |